

# Project 0: Unix/Python Tutorial

---

## Introduction

---

This project will cover the basics of working in the Unix environment and a small introduction to Python. In this course, we will only be using **Python 3**. You may remotely connect to the [SOE servers](#) or use lab workstations as well as your own computer.

## Submission

---

To get you familiarized with the automatic grading system, we will ask you to submit answers for problems 1 ( [buyLotsOfFruit](#) ) and 2 ( [shopSmart](#) ). The project may seem simple, but this is a good thing: learning the basics of Python now will save you many headaches later in the course. Take this easy project as an opportunity to **learn Python**.

Here is the code provided for this assignment: [p0.zip](#).

This assignment should be submitted with the filename `solution.zip` [HERE](#). Please use the command line `zip` tool to make sure your submission gets zipped correctly:

```
zip solution.zip buyLotsOfFruit.py shopSmart.py
```

For these submissions, unzip should directly yield the source files and **not** a directory named "solution" or "pN". Note that this is counter to standard conventions when sending a zip file to a human, but easier for the autograder.

After submissions, the autograder will grade your submission and report the result back to you. Do not close the tab or you will not be able to see your score. You can submit as many times as you want. However, if we find you continually making tiny changes instead of testing locally, then we will deduct points. Any attempts to trick the autograder is considered cheating.

## Resources

---

There are many great resources online for learning Unix and Python.

### UNIX

- [UNIX Tutorial for Beginners](#)
- [UNIX Tutorial](#)

### Text Editors

Being familiar with a powerful text editor can make a surprising difference. In the programming world, there are two text editors that have stayed on the top for over 30 years. Vi (Vim) and Emacs are two very powerful terminal-based text editors (although both have GUI versions). The arguments about which one is better have been going on for decades and will never be resolved. In truth, they are both amazing editors and will greatly increase your productivity once you learn how to use them. There are also many online resources for each:

- Vim
  - [Interactive Vim Tutorial](#)
  - [Visual Vim Tutorial](#)
  - [Vim Videos](#)
  - [Game-based Vim Tutorial](#)
- Emacs
  - [Practical Emacs Tutorial](#)
  - [Another Emacs Tutorial](#)
  - [Visual Emacs Tutorial](#)

### Git

Git is currently the most popular [Version Control System](#), period. Learning how to use source control is a must for all programmers in today's world. We **strongly** encourage all students to use source control for

their coursework. Of course, keep your repositories private so other students cannot see. Here are several resources for learning how to use git:

- [The full Pro Git book](#)
- [Git for Computer Scientists](#)
- [A Visual Git Reference](#)
- [A git Primer](#)

## Python

Python is one of the [most popular](#) programming languages. So, there are many online resource available for it. Here are just a few useful resources:

- [The full "Think Python" Book](#)
- [Official Tutorial](#)
- [Official Beginning Guide](#)
- [Hitchhiker's Guide to Python](#)
- [Data Structures in Python \(Official\)](#)
- [Data Structures in Python \(Unofficial\)](#)
- [List Comprehensions in Python](#)
- [Lambda Functions in Python](#)

## Assignment

---

For this assignment, you will be filling in two methods in two different Python files.

### Problem 1: Buy Lots Of Fruit

---

Implement the `buyLotsOfFruit` function in the `buyLotsOfFruit.py` file. This function will take a list of `(fruit, weight)` tuples and returns the cost of your list. If there is some fruit in the list which doesn't appear in `FRUIT_PRICES`, your function should print an error message and return `None` (which is like `null` in C and Java). Please do not change the `FRUIT_PRICES` dict.

### Problem 2: Shop Smart

---

Implement the `shopSmart` function in the `shopSmart.py` file. This function takes two arguments:

1. `orderList` - Like the kind passed in to `buyLotsOfFruit()`.
2. `fruitShops` - A list of `FruitShop`.

This function returns the `FruitShop` where your total order costs the least amount. Don't change the file name or variable names, please. Note that we will provide the `shop.py` implementation as a "support" file, so you don't need to submit yours.

## Frequent Errors

---

Here are some common problems (and their solutions) that new Python students have:

### ImportError - .py

#### Problem:

You get the following error when running your program:

```
ImportError: No module named py
```

#### Solution:

When using `import`, do not include the `".py"` from the filename. For example, you should say: `import shop` **not:** `import shop.py`.

### NameError - not defined

#### Problem:

You get the following error when running your program:

```
NameError: name 'my_variable' is not defined
```

### **Solution:**

To access a member of a module, you have to type `module_name.member_name`, where `module_name` is the name of the `.py` file, and `member_name` is the name of the variable (or function) you are trying to access.

### **TypeError - dict not callable**

#### **Problem:**

You get the following error when running your program:

```
TypeError: 'dict' object is not callable
```

#### **Solution:**

Dictionary lookups are done using square brackets: `[` and `]`. NOT parenthesis: `(` and `)`.

### **ValueError - too many to unpack**

#### **Problem:**

You get the following error when running your program:

```
ValueError: too many values to unpack
```

#### **Solution:**

Make sure the number of variables you are assigning to a variable or in a `for` loop matches the number of elements in each item of the list. Similarly for working with tuples.

For example, if `pair` is a tuple of two elements (e.g. `pair = ('apple', 2.0)`) then the following code would cause the "too many values to unpack error":

```
pair = ('apple', 2.0)
(a, b, c) = pair
```

Here is a problematic scenario involving a `for` loop:

```
pairList = [('apples', 2.00), ('oranges', 1.50), ('pears', 1.75)]
for fruit, price, color in pairList:
    print('%s fruit costs %f and is the color %s' % (fruit, price, color))
```

### **AttributeError - list length**

#### **Problem:**

You get the following (or similar) error when running your program:

```
AttributeError: 'list' object has no attribute 'length'
```

#### **Solution:**

Finding length of lists is done using `len(myList)`.